Empirical Analysis of Smart Contract Factories on EVM-compatible Chains

ZIYUE WANG, ZONGWEN SHEN, and LEI CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, China
WEI SONG, Nanjing University of Science and Technology, China
JIDONG GE*, State Key Laboratory for Novel Software Technology, Nanjing University, China
LIGUO HUANG, Department of Computer Science, Southern Methodist University, USA
BIN LUO, State Key Laboratory for Novel Software Technology, Nanjing University, China

Decentralized applications on EVM-compatible blockchains are powered by smart contracts—reprogrammable logic deployed on-chain. Among them, smart contract factories represent a distinct class of contracts that automate the creation of other contracts through `CREATE`/`CREATE2`, enabling scalable and repeatable deployments of tokens, protocol modules, and NFT collections. Factories are now the dominant vehicle for contract deployment and mint the majority of contracts. Yet, the ecosystem still lacks (i) a global view of their prevalence and application domains, (ii) a catalogue of implementation families beyond upgradeable proxies, and (iii) guidance on factory-specific security pitfalls that ripple across deployment pipelines.

To address these gaps, we present the first ecosystem-scale measurement of factory activity on two EVM-compatible chains: Ethereum and Polygon. We implement a bytecode-based factory contract detector that disassembles 434 million deployed runtimes, builds control-flow graphs, and uses reachability analysis to confirm on-path `CREATE`/`CREATE2` operations. This yields 120,204 Ethereum factories and 69,258 Polygon factories. The resulting dataset enables three key findings: (i) longitudinal measurements show factories have minted over 90% of contracts since 2020, concentrated in a small cadre of high-volume deployers; (ii) clustering 3,000 verified factories reveals four dominant application domains spanning Finance & token, Infrastructure & Protocol, Proxy & upgrade, and NFT & creator use cases; and (iii) semantic inspection distills six recurring implementation patterns alongside factory-specific attack vectors and security issues. Altogether, these results contribute to a deeper understanding of the current status, implementation patterns, and security issues of factory-based deployments.

CCS Concepts: • **Security and privacy → Distributed systems security**.

Additional Key Words and Phrases: Solidity, smart contract factory, factory pattern, empirical analysis.

*Corresponding author.

This work is supported by the National Natural Science Foundation of China Grants Nos. (92582204, 62472221), and the 6th "333 Project" Leading Talent Team Project of Jiangsu Province.
Authors' Contact Information: Ziyue Wang, ziyue0530@gmail.com; Zongwen Shen, szw@smail.nju.edu.cn; Lei Chen, willisgarey89@gmail.com, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China; Wei Song, wsong@njust.edu.cn, Nanjing University of Science and Technology, Nanjing, Jiangsu, China; Jidong Ge, gjd@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China; Liguo Huang, lghuang@lyle.smu.edu, Department of Computer Science, Southern Methodist University, Dallas, Texas, USA; Bin Luo, luobin@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/1-ART
https://doi.org/10.1145/3787216

Just Accepted

1 Introduction

Smart contracts [21] are blockchain programs widely used in decentralized finance (DeFi) and the Internet of Things (IoT), forming key Web3 infrastructure. As one of the most prominent blockchain platforms, Ethereum [13] supports smart contract deployment and execution, hosting over 66 million Solidity contracts as of January 2025 [22].

In recent years, the rise of complex decentralized applications (DApps) has made contract deployment and management crucial [32, 51, 61, 65]. However, manually writing and deploying contracts is repetitive and risky, making it unsuitable for large-scale contract deployment. To address this issue, developers increasingly deploy contracts with the help of smart contract factories [7] as advocated by Ethereum improvement proposals (EIPs) [1–3, 10] and third-party libraries [4, 8, 9, 11, 16]. The factory approach ensures efficient and scalable deployment and provides solutions to concerns such as contract upgradeability and gas optimization.

Factory-mediated deployment is now ubiquitous across major EVM-compatible chains. Aggregating Ethereum and Polygon before June 1, 2025, factory contracts are responsible for roughly 19% of the deployed contracts that we observe, and on the busiest days they account for more than half of the new deployments implemented through CREATE or CREATE2 opcodes. This sustained footprint underscores the need for a systematic understanding of how factory contracts are adopted, engineered, and secured across ecosystems. Despite this prominence, the research landscape remains fragmented. **(i) Missing ecosystem-scale evidence on factory prevalence and domains.** Existing measurements concentrate on narrow slices—e.g., contract clustering studies [41, 55, 59] or inline-assembly analyses [30]—and therefore miss cross-chain deployment volume and sector diversity. **(ii) Missing comprehensive views of factory implementation patterns beyond proxies.** Prior work on proxy ecosystems [45, 64] and upgradeable design guidelines [1–3, 10] explores individual patterns but stops short of mapping the broader family of factory templates used in production. **(iii) Missing factory-aware analysis of emerging attack vectors and issues.** General vulnerability analyzers [29, 43, 48, 52, 62] and proxy-focused investigations [45, 55] rarely reason about factory-specific semantics such as deployer provenance, CREATE/CREATE2 interplay, or metamorphic contract workflows, leaving the practical guidance fragmented.

To operationalize the study, we extract 434,542,165 deployed contracts (3,680,947 unique runtimes) from Ethereum and Polygon via Google BigQuery, then run a bytecode-based Factory Contract Detector that disassembles each runtime, builds a control-flow graph, and performs reachability analysis to confirm on-path CREATE/CREATE2 opcodes before flagging factories. This pipeline surfaces 120,204 Ethereum factories and 69,258 Polygon factories, which anchor our measurements of deployment prevalence, application domains, implementation templates, and security posture. We structure the investigation around five research questions. **RQ1** evaluates the detector against curated ground truth to secure the methodological foundation for all subsequent analyses. **RQ2** quantifies temporal prevalence and deployment throughput across chains, addressing gap (i). **RQ3** profiles factory application domains via verified source-code clustering, further addressing gap (i). **RQ4** maps implementation templates and functionalities from representative factories to close gap (ii). **RQ5** examines factory-induced attack vectors and governance failures, closing gap (iii).

Our key findings—which span five research questions—include:

- The factory contract detector achieves 0.17% false-positive rate and 8.39% false-negative rate on a 2,907-contract ground truth while processing 75% of samples within 24.2 ms; the remaining false negatives stem from proxy-mediated factories whose CREATE/CREATE2 opcodes reside in delegated implementations beyond bytecode reachability.
- Ethereum's 120,204 and Polygon's 69,258 factories jointly drive more than 84 million deployments, mint over 90% of contracts since 2020, and exhibit heavy concentration where the top 0.04% of factories issue 80% of CREATE/CREATE2 traces.

Just Accepted

- Clustering 3,000 verified factories surfaces four dominant domains—Finance & token (63.6%), Infrastructure & protocol (18.5%), Proxy & upgrade (15.4%), and NFT & creator (2.6%)—capturing ecosystem-scale factory adoption.
- Semantic analysis distills six recurring implementation patterns (Template Configuration, Proxy Delegation, Centralized Registry, Keyless Singleton, Salted Address, Metamorphic) that articulate the engineering idioms underpinning high-volume factories.
- The security assessment uncovers two factory-specific attack vectors and three security issues that manifest in real deployments, motivating mitigations for governance and deployment.

Contributions. In summary, this paper makes the following contributions:

- **Systematic Panorama.** We deliver the first ecosystem-scale, multi-chain study of smart-contract factories, spanning prevalence, application domains, implementation practices, and security risks across Ethereum and Polygon.
- **Detection Infrastructure.** We build a bytecode-based factory contract detector and curated evaluation dataset, enabling reproducible collection of factory deployments and supporting follow-on analyses and tooling.
- **Actionable Insights.** We distill interpretable patterns, domain characterizations, and security guidance that inform developers, auditors, and governance communities working on factory-based deployments.

Data Availability. All artifacts, including detector implementation and factory datasets, are released at https://anonymous.4open.science/r/tosem_fsc.

The rest of the paper is organized as follows. We first introduce the background in Section 2. Then we detail the methodology and present findings in Section 3. We further discuss our results in Section 4, summarize related work in Section 5, and conclude in Section 6.

2 Background Knowledge

In this section, we introduce the background knowledge that is useful for understanding our research.

2.1 Smart Contracts

Smart contracts [21] are self-executing contracts with the terms of the agreement directly written into code. These contracts are designed to automatically enforce and execute predefined rules and conditions, eliminating the need for trusted intermediaries and fostering transparency in transactions. While various smart contract programming languages exist, Solidity [15] is the predominant language used for developing smart contracts on Ethereum and other Ethereum Virtual Machine (EVM)-compatible blockchains. A Solidity smart contract comprises several key components, including:

- **State Variables**: These variables store the contract's persistent data, representing its current state. They hold information such as account balances, ownership records, and configuration parameters.
- **Functions**: Functions define the contract's behavior and logic. They are executable units of code that can be called by external users or other smart contracts to interact with the contract's state.
- **Modifiers**: Modifiers are code blocks that can be attached to functions to enforce pre-conditions. They provide a mechanism for access control, validation, and other security checks before a function can be executed. For example, a common modifier is onlyOwner, which restricts function execution to the contract owner.
- **Events**: Events allow smart contracts to log actions and state changes that occur during execution. These logs can be monitored by external applications to track contract activity and trigger off-chain processes.

Just Accepted

2.2 Ethereum and Etherscan

Ethereum [13] represents a significant advancement in blockchain technology, serving as a pioneering public, permissionless, and distributed computing platform. While building upon the foundational principles established by Bitcoin [23], Ethereum expands the application of blockchain beyond a purely peer-to-peer digital currency system. It achieves this by enabling the deployment and execution of smart contracts, programmable agreements that automatically execute when predefined conditions are met. This capability has unlocked a new paradigm for decentralized applications (dApps) across a wide range of industries.

At the heart of Ethereum lies the Ethereum Virtual Machine (EVM) [25], a Turing-complete runtime environment that executes the bytecode of smart contracts. The EVM provides a deterministic and isolated execution environment, ensuring that contracts behave predictably and securely, regardless of the underlying hardware. The EVM's architecture is crucial for maintaining the integrity and consistency of the Ethereum blockchain. Gas [27], a unit of computation, is used to measure the computational cost of executing operations on the EVM, preventing denial-of-service attacks and ensuring fair resource allocation.

Etherscan [14] is a prominent blockchain explorer and analytics platform specifically designed for the Ethereum network. It provides a user-friendly interface for accessing and analyzing data stored on the Ethereum blockchain, including transactions, smart contracts, addresses, and blocks. Etherscan's functionalities are essential for developers, researchers, and users seeking to understand the workings of the Ethereum ecosystem. Key features of Etherscan include:

- **Transaction Tracking**: Users can track the status and details of individual transactions, including sender, recipient, gas used, and transaction fee.
- **Smart Contract Inspection**: Etherscan allows users to view the source code of verified smart contracts, enhancing transparency and enabling security audits.
- **Address Analysis**: Users can analyze the transaction history and balance of specific Ethereum addresses, gaining insights into their activity on the network.

2.3 Account Types

EVM-compatible blockchains distinguish between two fundamental types of accounts that participate in blockchain operations.

- **Externally Owned Accounts (EOAs).** An EOA [24] is a type of account controlled by a private key, capable of initiating transactions on the blockchain. EOAs serve as the primary interface for users to interact with the blockchain network and can hold Ether balances.
- **Contract Accounts.** Contract accounts contain smart contract code and are created through deployment transactions. Unlike EOAs, contract accounts do not have private keys and can only execute transactions when triggered by external calls from EOAs or other contracts. Contract accounts can hold Ether balances and maintain persistent storage state.

2.4 Smart Contract Deployment Methodologies

This subsection introduces the deployment mechanisms and related concepts for smart contracts on EVM-compatible blockchains.

- **EOA-deployed Contracts.** These are smart contracts deployed through direct transactions initiated by EOAs. In this deployment mechanism, an EOA issues a create-type transaction containing the contract's bytecode, resulting in the direct deployment of the contract on-chain.
- **Factory Contracts.** A factory contract [7] is a specialized smart contract designed to programmatically deploy other contracts. Factory contracts contain deployment logic and execute internal create or create2 transactions to instantiate new contract instances. Examples include the UniswapV2Factory used by

Just Accepted

Uniswap [18] to deploy trading pair contracts, and ERC20Factory contracts provided by platforms like ThirdWeb [17] for token generation.

- **Factory-deployed Contracts.** These are smart contracts created through factory contract mechanisms. The deployment process involves an EOA invoking a factory contract's deployment function, which subsequently executes the actual contract creation through internal transactions.

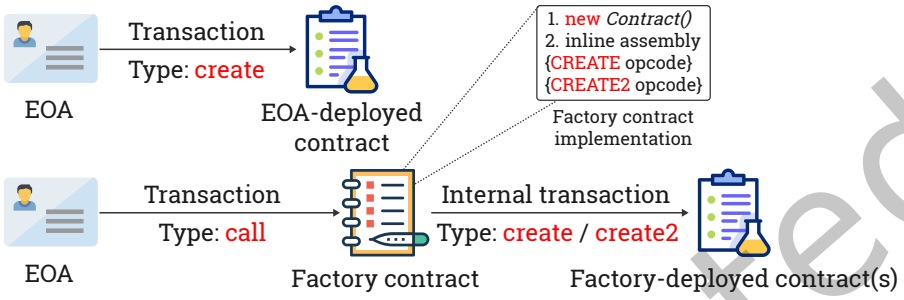

Fig. 1. Comparison of two smart contract deployment methodologies on EVM-compatible chains. (i) Direct EOA deployment: Externally Owned Accounts (EOAs) issue create-type transactions to deploy contracts directly on-chain. (ii) Factory-mediated deployment: EOAs invoke factory contracts through call-type transactions, which then execute internal create/create2 transactions to deploy target contracts. Factory contracts utilize two primary implementation approaches: high-level *new Contract()* syntax and low-level inline assembly with CREATE/CREATE2 opcodes.

Just Accepted

Figure 1 illustrates the comparison between contracts deployed by Externally Owned Accounts (EOA) [24] and those deployed through a factory contract. In a direct EOA deployment, an EOA initiates a create transaction, directly deploying a new contract to the blockchain. Conversely, factory deployment involves a two-stage process. First, an EOA sends a transaction to the factory contract, invoking a specific method designed to handle contract deployment. This transaction includes necessary parameters such as constructor arguments for the new contract, Ether (ETH) for initialization, and potentially a salt value, depending on the deployment method. The factory contract then initiates an internal transaction, using either the *create* or *create2* opcode, to execute the actual contract creation and deployment.

The underlying mechanism of smart contract factories relies on two core EVM opcodes: *create* and *create2*. The *create* opcode determines the deployment address using the formula *addr = keccak256(rlp(sender, nonce))*, where sender represents the address of the account (in this case, the factory contract) creating the new contract, and nonce is the number of transactions sent by that account. This approach results in deployment addresses that are dependent on the factory's transaction history. The *create2* opcode, introduced through EIP-1014 [1], offers a deterministic and more predictable deployment address calculation: *addr = keccak256(0xff, sender, salt, keccak256(init_code))*. Here, sender is the address of the factory contract, salt is a user-defined value (allowing for deterministic deployment), and *init_code* is the initialization code of the contract being deployed. Developers can trigger contract creation using the high-level *new Contract()* statement within Solidity, which typically compiles to the *create* opcode. Alternatively, developers can directly use these opcodes within inline assembly blocks for finer-grained control over the deployment process.

3 Empirical Study

This section presents our empirical study, including the approach and results.

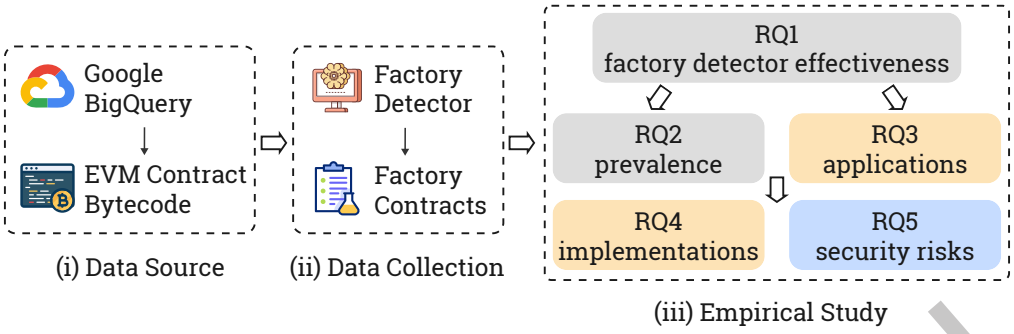

Fig. 2. Overall workflow of the empirical study across two EVM-compatible chains. (i) Contract bytecodes deployed before June 1st, 2025 are extracted from two EVM-compatible chains (Ethereum and Polygon) using Google BigQuery. (ii) factory contract detector analyzes all bytecodes to identify and collect factory contracts. (iii) The empirical investigation follows five sequential research questions, where RQ1 serves as the foundation for subsequent analyses. Gray blocks indicate studies based on contract bytecode (RQ1 and RQ2), orange blocks represent source code-based analyses (RQ3 and RQ4), and blue blocks denote investigations requiring both bytecode and source code (RQ5).

Just Accepted

3.1 Research Questions

Our study aims to address five research questions that systematically investigate factory-related smart contracts across two major EVM-compatible chains.

- **RQ1: Detector Effectiveness**. We evaluate the effectiveness of our factory contract detection methodology as the foundation for subsequent analyses. *How accurately can we identify factory contracts from bytecode? What is the performance of our detection approach?*
- **RQ2: Contract Prevalence**. We quantify the prevalence and deployment trends of factory contracts across EVM-compatible chains. *How prevalent are factory contracts in real-world deployments? What are the temporal trends in factory contract deployment? How do create and create2 opcode usage patterns evolve over time?*
- **RQ3: Application Domains**. We investigate the primary application domains and use cases where factory contracts are deployed. *What are the major application domains of factory contracts? How are factory contracts distributed across different blockchain sectors? What drives the adoption of factory patterns in specific domains?*
- **RQ4: Implementation Analysis**. We explore the technical implementation patterns and functionalities of factory contracts. *What unique functionalities do factory contracts possess? What are the prevalent design patterns in factory implementations? How do different implementation approaches affect contract deployment efficiency?*
- **RQ5: Attack Vector and Security Issue Analysis**. We examine the security risks and vulnerabilities associated with factory-related smart contracts. *What are the key attack vectors and security issues in factory contracts? How can these attack vectors and vulnerabilities be systematically detected? What are the potential impacts and mitigation strategies for identified risks?*

3.2 Empirical Study Overview

As illustrated in Figure 2, our empirical study follows a systematic three-phase approach to investigate factory-related smart contracts across two EVM-compatible chains.

Table 1. Dataset Overview

Chain	Total Contract	Unique Bytecode
Ethereum	73,390,233	1,672,444
Polygon	361,151,932	2,008,503
Total	434,542,165	3,680,947

Phase I: Data Source. We utilize Google BigQuery [34] as our primary data source to extract contract bytecodes from two major EVM-compatible chains: Ethereum [13] and Polygon [53]. We specifically selected these two chains because they are the only EVM-compatible chains in Google BigQuery where all contract bytecodes are comprehensively available in the BigQuery Public Dataset [33]. This comprehensive dataset provides the foundation for our analysis.

Phase II: Data Collection. Our factory contract detector analyzes the collected bytecodes to systematically identify and extract factory contracts from the multi-chain dataset. This automated detection process ensures consistent identification criteria across both blockchain chains.

Phase III: Empirical Study. The investigation follows five sequential research questions, where RQ1 (factory detector effectiveness) serves as the methodological foundation for subsequent analyses. RQ2-RQ5 examine prevalence, applications, implementations, and security risks respectively, utilizing both bytecode-based and source code-based analytical approaches.

3.3 Data Source

We extract all contract deployed bytecodes[1] deployed before June 1st, 2025 from Ethereum (mainnet) [13] and Polygon [53]. As shown in Table 1, our dataset comprises 434,542,165 total contracts with 3,680,947 unique bytecodes across the two chains, with Ethereum contributing 73,390,233 contracts (1,672,444 unique bytecodes) and Polygon contributing 361,151,932 contracts (2,008,503 unique bytecodes).

3.4 Factory Contract Detector

Our factory contract detector employs control flow graph (CFG) analysis on deployed bytecode to accurately identify factory smart contracts. The detector's core algorithm is presented in Algorithm 1. Specifically, the detector constructs a control flow graph from the deployed bytecode by first disassembling it into EVM instructions and identifying basic blocks. Each basic block represents a sequence of instructions with a single entry and exit point. The algorithm then performs reachability analysis from the contract's entry point to determine which basic blocks are executable during contract execution. Finally, the detector examines only reachable basic blocks for *CREATE* and *CREATE2* operations, ensuring that contract deployment instructions found in data segments or unreachable code paths do not result in false positive detections.

3.5 Factory Contract Collection

Our factory contract collection system systematically gathers factory contracts from two EVM-compatible blockchains. Each contract is analyzed using our detector, with results stored including factory classifications (Non-factory, CREATE-only, CREATE2-only, or Both) and execution time.

[1]Deployed bytecode refers to the contract's runtime bytecode stored on-chain after deployment, excluding constructor code and initialization parameters.

Just Accepted

3.6 RQ1: Detector Effectiveness

This research question aims to evaluate the effectiveness of our factory contract detector through metrics including precision, recall, and execution efficiency. Validating the detector's effectiveness is crucial as it ensures the accuracy and reliability of our factory contract identification, which forms the foundation for all subsequent analyses (RQ2-RQ5).

Ground-Truth Dataset Construction. We construct a ground-truth dataset to evaluate detector effectiveness through two complementary approaches. (i) For factory contracts, we utilize Google BigQuery's Ethereum *traces* table [35] to identify contracts that executed CREATE or CREATE2 operations in non-constructor contexts with successful status, ensuring definitive factory classification through on-chain execution records. (ii) For non-factory contracts, we obtain verified smart contracts from Etherscan [60] and filter those whose source code contains no "new", "create", or "create2" keywords, guaranteeing non-factory classification. This methodology yields 548 factory contracts with unique bytecode and 2359 non-factory contracts with unique bytecode for evaluation.

False Positive Rate and False Negative Rate. To evaluate the performance of our factory detector, we define the following classification categories:

- **TP (True Positive):** The current contract is a factory contract, and the factory detector correctly identifies it as a factory contract.
- **FP (False Positive):** The current contract is not a factory contract, but the factory detector incorrectly classifies it as a factory contract.
- **FN (False Negative):** The current contract is a factory contract, but the factory detector fails to identify it as such.
- **TN (True Negative):** The current contract is not a factory contract, and the factory detector correctly identifies it as a non-factory contract.

We calculate the False Positive Rate (FPR) and False Negative Rate (FNR) metrics to assess our detector's effectiveness. FPR = FP/(FP+TN) measures the proportion of non-factory contracts incorrectly classified as factories, while FNR = FN/(FN+TP) measures the proportion of factory contracts missed by the detector.

Our evaluation on the complete dataset of 2,907 contracts demonstrates exceptional performance with a False Positive Rate of **0.17%** and a False Negative Rate of **8.39%**. Specifically, out of 548 factory contracts, our detector correctly identifies 502 (TP = 502) while missing 46 (FN = 46). Among 2,359 non-factory contracts, our detector correctly classifies 2,355 (TN = 2,355) but incorrectly flags only 4 as factories (FP = 4). The extremely low False Positive Rate indicates that our detector rarely misclassifies non-factory contracts, ensuring high reliability in factory identification. The moderate False Negative Rate primarily stems from proxy-based factory patterns that exceed the scope of static bytecode analysis; we analyze this further in subsequent analyses.

False Negative Analysis. Our analysis of the 46 false negatives reveals that all misclassified cases are attributable to proxy-based factory patterns. These contracts delegate their factory functionality to implementation contracts through proxy mechanisms, where the actual CREATE/CREATE2 operations occur in the delegated bytecode rather than the proxy contract's own bytecode. The execution traces confirm that these contracts successfully performed factory operations, but our static bytecode analysis cannot directly obtain the addresses of the latest implementation contracts associated with proxies, which exceeds the scope of bytecode-level detection. This limitation represents the primary constraint of our detector: the inability to analyze proxy-based factory architectures where factory functionality is abstracted through delegation patterns.

False Positive Analysis. Our analysis of the 4 false positives demonstrates exceptionally high precision in factory detection. These misclassifications result from our detector erroneously interpreting embedded data sequences within the bytecode as CREATE / CREATE2 opcodes.

Just Accepted

Algorithm 1 factory contract detector

Input: Contract deployed bytecode *bytecode*
Output: Factory detection result with verification details

```
procedure DETECTFACTORYCONTRACT(bytecode)
    instructions ← disassemble(bytecode)
    if instructions = ∅ then
        return NotFactory
    basic_blocks ← buildBasicBlocks(instructions)
    cfg ← addControlFlowEdges(basic_blocks, instructions)
    reachable ← performReachabilityAnalysis(cfg, entry_point = 0)
    verified_creates ← ∅
    verified_create2s ← ∅
    for all block ∈ reachable do
        (creates, create2s) ← extractCreateOperations(block)
        verified_creates ← verified_creates ∪ creates
        verified_create2s ← verified_create2s ∪ create2s
    if verified_creates ≠ ∅ or verified_create2s ≠ ∅ then
        return FactoryContract
    else
        return NotFactory
```

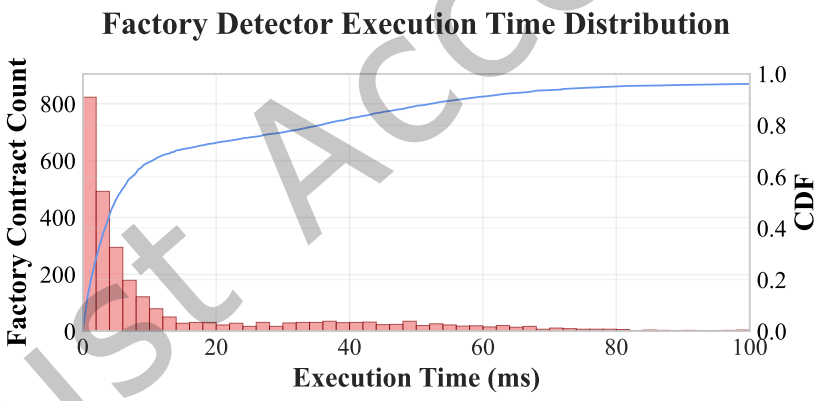

Fig. 3. Cumulative Distribution Function (CDF) of execution time for factory contract detection across the ground-truth dataset.

Just Accepted

Execution Performance. Our factory detector demonstrates excellent computational efficiency across the entire ground-truth dataset, as shown in Figure 3. The execution time distribution reveals that the majority of contracts are analyzed rapidly, with the histogram showing peak frequency in the low-latency range. The cumulative distribution function indicates that 25% of contracts complete analysis within 1.7 milliseconds, while 50% achieve detection in 4.7 milliseconds or less. Furthermore, 75% of all contracts are processed within 24.2 milliseconds, demonstrating the scalability of our approach for large-scale contract analysis.

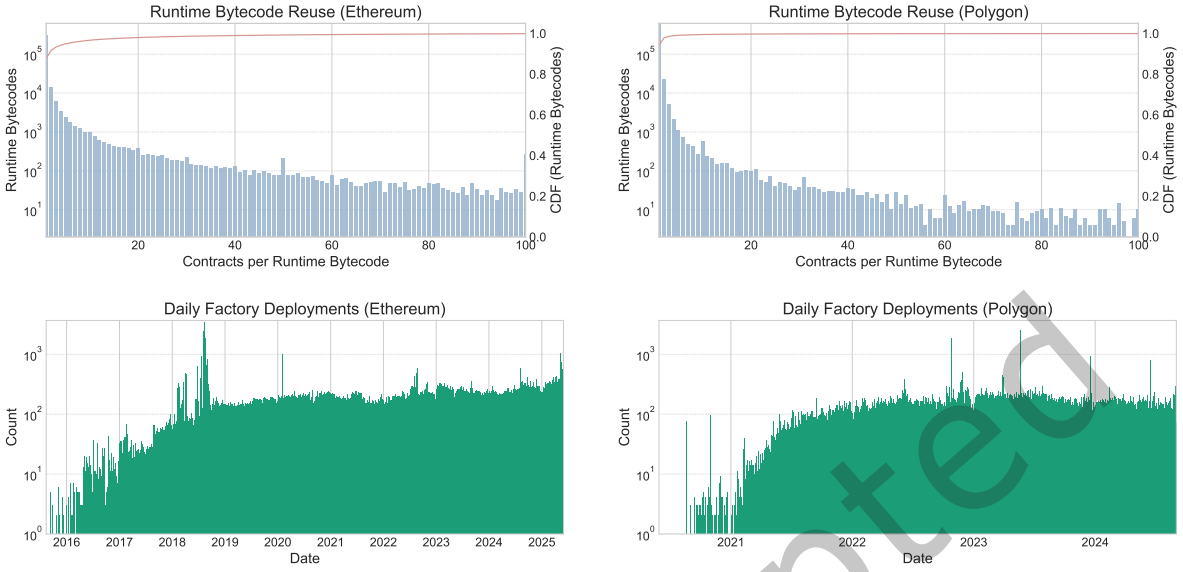

Fig. 4. RQ2 (i) and (ii) comparison. Top row: Runtime bytecode reuse distribution (left: Ethereum, right: Polygon). X-axis: number of contracts per runtime bytecode (occurrences). Y-axis: number of distinct runtime bytecodes with that occurrence count. The overlaid line shows the cumulative distribution (CDF). Bottom row: Daily active factories. X-axis: date. Y-axis: count of unique factory contracts that executed at least one CREATE/CREATE2 on that day.

Just Accepted

Answer to RQ1: Our factory contract detector achieves exceptional effectiveness with a False Positive Rate of **0.17%** and False Negative Rate of **8.39%** on 2,907 contracts. The detector correctly identifies 502/548 factory contracts while misclassifying only 4/2,359 non-factory contracts. False negatives stem from proxy-based patterns beyond static analysis scope; false positives result from misinterpreting embedded data as CREATE/CREATE2 opcodes. Performance is excellent with 75% of contracts analyzed within 24.2 milliseconds.

3.7 RQ2: Prevalence Investigation

(i) Runtime bytecode reuse. Both chains show extreme reuse with a long tail (Figure 4, top). On Ethereum there are **362,813** distinct runtime bytecodes; **86.61%** of them (314,268/362,813) appear only once, covering merely **0.43%** of factory-created contracts (314,268/72,881,396). In contrast, runtime bytecodes with reuse ≥ 100 cover **98.89%** of contracts, and just **0.30%** of runtime types (reuse ≥ 1000) cover **97.32%**. Polygon is similar: **626,702** runtimes in total; **94.06%** appear once yet cover only **5.36%** of contracts; reuse ≥ 100 covers **92.97%**, and only **0.04%** of runtime types (225/626,702, reuse ≥ 1000) cover **91.06%**. A tiny set of templated runtimes dominate the vast majority of factory-created contracts.

(ii) Daily active factories. As shown in Figure 4 (bottom), both chains have stable medians with occasional spikes. Ethereum has activity on **3,476** days (2015-08-13 to 2025-05-31), averaging **152** active factories per day (median **160**, 95th **286**); the maximum is **3,342** on 2018-08-10. Polygon spans **1,421** days (2020-05-30 to 2024-09-01), averaging **126** (median **127**, 95th **221**); the maximum is **2,526** on 2023-05-22.

(iii) Daily factory-issued CREATE/CREATE2 deployments. Figure 5 shows that Ethereum factories mint **20,967** contracts per day on average through internal CREATE/CREATE2 traces (median **10,662**, 95th **76,617**),

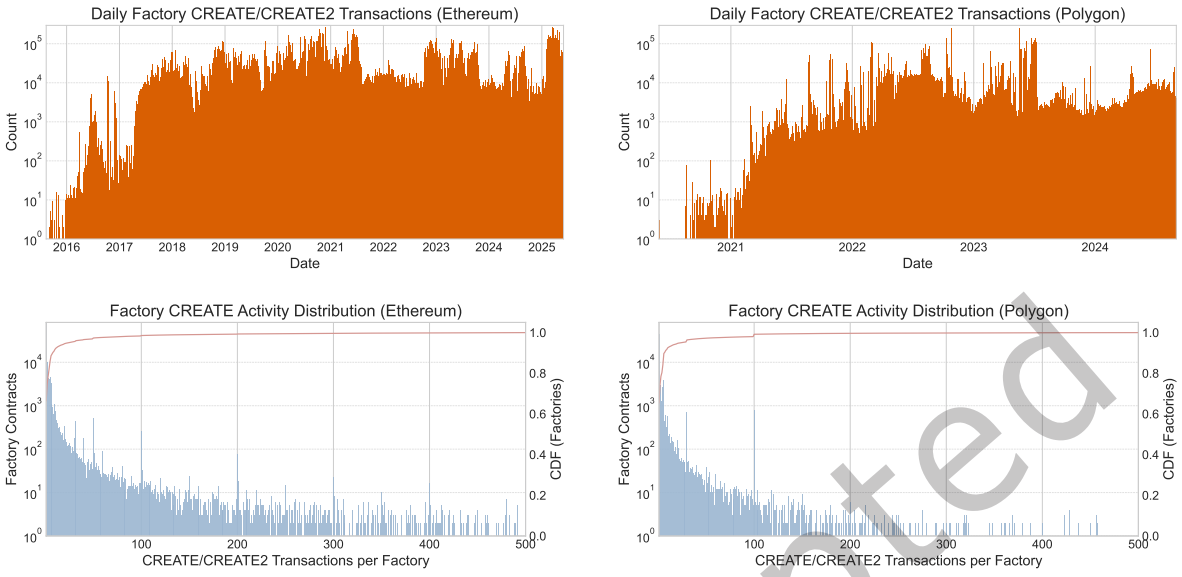

Fig. 5. RQ2 (iii) and (iv) comparison. Top row: Daily factory-issued CREATE/CREATE2 deployments. X-axis: date. Y-axis: number of internal CREATE/CREATE2 traces (contracts minted by factories) on each day. Bottom row: Per-factory cumulative CREATE/CREATE2 distribution. X-axis: total CREATE/CREATE2 per factory (lifetime). Y-axis: number of factories. The overlaid line shows the cumulative distribution (CDF).

Just Accepted

peaking at **267,857** new contracts on 2020-11-29. Polygon factories average **8,200** deployments per day via CREATE/CREATE2 (median **2,674**, 95th **32,506**), with a peak of **250,203** on 2022-10-26.

(iv) Per-factory cumulative activity. Figure 5 (bottom) summarizes lifetime CREATE/CREATE2 per factory. On Ethereum there are **120,204** factories; **65.56%** created exactly once, **90.47%** created at most 10 times, and **2.97%** created ≥ 100 times. The mean is **606** creates/factory, yet creation volume is highly concentrated: the top **50** factories (**0.04%**) account for **80%** of creates. Polygon has **69,258** factories; **65.07%** created once, **91.99%** at most 10, **2.50%** ≥ 100; mean **168**. The top **21** factories (**0.03%**) cover **80%** of creates.

Answer to RQ2: Factory deployments are highly templated and concentrated. A tiny fraction of runtime types ($\leq$ **0.30%**) covers **91–97%** of factory-created contracts on Ethereum and Polygon, while most runtimes are singletons with negligible coverage. Daily activity is steady with substantial throughput (ETH median **160** factories and **10,662** creates/day; POLY **127** and **2,674**). Creation is extremely skewed: the top **50** (ETH) and **21** (POLY) factories produce **80%** of creates.

3.8 RQ3: Application Domains

We sample verified factory contracts from the RQ2 factory table: addresses are sorted by factory activity and we take the top **3,000** with verified source contracts. From each Solidity contract we extract multi-view features capturing code semantics and mechanisms, including: (i) name tokens from declared contract names; (ii) size/structure signals (lines of code, numbers of contracts/interfaces, constructors, modifiers, and imports); (iii) mechanism flags (create/create2/clone/ predict/salt/delegatecall); (iv) access-control and library usage (Ownable/AccessControl/onlyOwner, OpenZeppelin and common libraries via using-for directives); (v) business keywords (dex, token,

Table 2. Clustering results for application domains (K=4).

Category	Keywords / Signals	Number
Finance & Token	token, ico, sale, vesting, dividend, fee, vault, stake, yield, distribution	1,909 (63.6%)
Infrastructure & Protocol	router, pool, pair, registry, manager, governance, dao, bridge, oracle, module	554 (18.5%)
Proxy & Upgrade	proxy, upgradeable, delegatecall, router, beacon, uups, clone, factory, create2	461 (15.4%)
NFT & Creator	nft, art, mint, creator, collection, royalty, metadata, game	76 (2.6%)

nft, wallet, dao, proxy, uups, beacon, etc.) and visibility ratios. We standardize features and apply PCA with Varimax rotation, then cluster the rotated components using KMeans (k=4). For visualization we embed the data with t-SNE, aggregate identical feature vectors, and hide the farthest 10% points per cluster for clarity. The combined map is shown in Figure 6 and the category summary in Table 2.

(i) Finance & token. Token issuance and distribution contracts (ICO/airdrop/vesting), fee/dividend distribution, staking/vault management. Beyond plain ERC-20 deployment, factories commonly parameterize name/symbol-/initial supply, fee switches (e.g., fee-on-transfer), treasury or fee recipients, and vesting schedules, and often spin up per-project payment splitters or timelocked vaults. Strong signals include token/erc vocabulary, Ownable/AccessControl, and templated create/clone deployments, together with distribution helpers (airdrop/vesting) and splitter patterns.

(ii) Infrastructure & protocol. Protocol scaffolding such as routers/pools/registries and per-pair or per-pool factories used by AMMs and protocol suites. Deployments emphasize deterministic addresses (via *create2*) for discoverability, registry-backed lookup, and reproducible per-market instances across chains. Characterized by create2/clone, manager/registry naming tokens, and usage of Clones/Create2 libraries; auxiliary modules include fee routers, treasuries, and keepers.

(iii) Proxy & upgrade. Factories that deploy proxy or minimal-proxy instances and manage upgrades. Typical stacks include EIP-1167 minimal proxies, EIP-1967 storage slots, UUPS/Beacon patterns, initializer guards, and upgrade-admin timelocks. Dominated by proxy/upgradeable/beacon/uups/*delegatecall* signals and upgrade controllers; init data is passed at creation, with admin/owner roles centralized in the factory.

(iv) NFT & creator. Factories for ERC-721/1155-style collections, generative art, and games. Common parameters cover baseURI, supply caps, mint price/currency, royalty settings (EIP-2981), operator filters, and withdrawal recipients. Signals include mint/collection/royalty/metadata keywords with simple access control and drop mechanics (allowlist/Merkle proofs, phased sales, airdrops), frequently accompanied by per-collection payment splitters and reveal toggles.

Answer to RQ3: Clustering the top-3,000 verified factory contracts with KMeans (k=4) yields four application domains: Finance & token (63.6%), Infrastructure & protocol (18.5%), Proxy & upgrade (15.4%), and NFT & creator (2.6%). These domains reflect the primary intents of factory usage across chains.

3.9 RQ4: Functionalities and Patterns Analysis

This study examines factory contract behaviors and implementation patterns. Since analyzing all contracts individually is costly, we focus on verified high-volume contracts and involve three smart contract security experts for analysis. We follow three systematic steps to uncover factory functionalities and implementation patterns: ① Factory Decomposition - We obtain five main smart contract components from Solidity documentation, including State Variables, Functions, Modifiers, Events, and Errors, with expert manual review ensuring each functionality correlates with multiple contract components. ② Factory-based Pattern Identification - We explore

Just Accepted

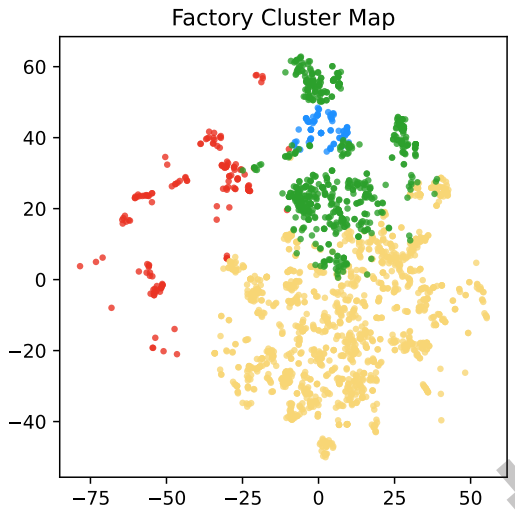

Fig. 6. Factory Cluster Map (K=4) of top-3,000 verified factories. Legend: green blocks = Finance & token; orange blocks = Infrastructure & Protocol; blue blocks = Proxy & upgrade; pink blocks = NFT & creator.

Just Accepted

patterns by examining contract semantics, motivations, and functionalities, sourcing Ethereum Improvement Proposals related to factory contracts with independent auditing and regular expert meetings. ③ Pattern Extraction - We use static analysis to detect patterns in the dataset based on syntactic and semantic features. Table 3 details 6 factory-based patterns, their motivations, utilized functionalities, and quantities observed in our dataset.

Template Configuration Pattern. This is the most fundamental pattern in factory contracts. It deploys multiple instances of contracts with identical runtime bytecode. The pattern dictates that the factory should deploy new contracts based on a predefined template and configure new instances via constructor parameters or initialization methods post-deployment. This pattern is widely utilized in various dApps, such as ERC-20 token contracts and ERC-721 NFT contracts. Two main implementation methods exist: using the *new* keyword (e.g., *new Token(strategy, expiration, decimal)*) or using *create* or *create2* in inline assembly. For instance, UniswapV2Factory [12] shown in Figure 7 uses *create2(0,add(),mload(),salt)* to create new pair contracts, followed by initialization with *IUniswapV2Pair(p).initialize(t0,t1)*.

Proxy Delegation Pattern. While the Template Configuration Pattern is a simple way to deploy multiple instances, storing the bytecode of each contract repeatedly can significantly increase gas costs. This pattern mitigates gas costs associated with storing repeated bytecode by deploying one master contract instance. All other instances act as proxies, delegating calls to the master while maintaining independent states.

We analyze the implementation methods and quantities of this pattern in our dataset. Many factories use @openzeppelin/contracts/proxy/Clones.sol to deploy minimal proxy contracts. Figure 8 shows the code snippet of the Clones contract, which follows EIP-1167 (Minimal Proxy Contract) [2]. This factory contract creates a minimal proxy contract of a specified implementation contract, which uses minimal bytecode to delegate calls to a fixed address, optimizing gas costs. However, unverified master contract addresses could pose security risks, discussed in Section 3.10.

Centralized Registry Pattern. This pattern enhances the management and traceability of deployed contracts by using a central registry contract to store the addresses of deployed contract instances along with their

Table 3. Identified Factory-based Patterns

Pattern Name	Motivation	Used Functionalities
Template Configuration Pattern	Facilitate efficient deployment and configuration of identical contracts	Use new or *create* / *create2* to deploy contracts. Then call init_function or pass parameters to the constructor for contract initialization
Proxy Delegation Pattern	Optimize gas costs by deploying minimal proxy contracts for identical contracts	Use Clones library to deploy minimized proxy contracts. Validate the untrusted master contract
Centralized Registry Pattern	Improve management efficiency and traceability of deployed contracts through a centralized approach	Use *map* / *array* to store and keep track of all deployed contracts. Check which contracts have been previously deployed
Keyless Singleton Pattern	Deploy contracts with the same init code to the same address in multiple blockchains	Use Nick's method to deploy keyless factories. Utilize *create2* for deterministic contract address independent of deployer's nonce
Salted Address Pattern	Calculating the contract address requires only the deployer's address and salt, excluding initcode hash	Utilize *create2* opcode for deterministic contract address independent of deployer account's nonce
Metamorphic Factory Pattern	Support contract upgradeability and some malicious purposes	Use *selfdestruct* opcode to clear contract code and storage. Use a transient factory to deploy metamorphic contracts

```
function createPair( address tokenA,  address tokenB) external returns ( address pair) {
   bytes memory bytecode = type(UniswapV2Pair).creationCode;
   bytes32 salt = keccak256(abi.encodePacked(token0, token1));
  assembly {
    pair :=  create2(0,  add(bytecode, 32),  mload(bytecode), salt)
  }
  IUniswapV2Pair(pair).initialize(token0, token1);
  getPair[token0][token1] = pair;
  getPair[token1][token0] = pair;
  allPairs.push(pair);
  emit PairCreated(token0, token1, pair, allPairs.length);
}
```

Fig. 7. Code snippet of the UniswapV2Factory contract, which uses the bytecode of UniswapV2Pair to deploy new trading pair contracts.

```
function cloneDeterministic( address implementation, bytes32 salt, uint256 value) internal{
  assembly ("memory-safe") {
    //Cleans the upper 96 bits of the `implementation` word, then packs the first 3 bytes of the `
        implementation` add
 ress with the bytecode before the  address.
    mstore(0x00, or(shr(0xe8, shl(0x60, implementation)), 0
        x3d602d80600a3d3981f3363d3d373d3d3d363d73000000))
    //Packs the remaining 17 bytes of `implementation` with the bytecode
    mstore(0x20, or(shl(0x78, implementation), 0x5af43d82803e903d91602b57fd5bf3))
    instance :=  create2(value, 0x09, 0x37, salt)
  }
  if (instance ==  address(0)) {
    revert Errors.FailedDeployment();
  }
}
```

Fig. 8. Code snippet of the OpenZeppelin Clones contract, this function creates a minimal proxy of the implementation contract.

associated information, such as version and owner details. As shown in Figure 9, a central registry maintains mappings of deployed proxies, enabling simplified querying and verification through centralized access control.

Just Accepted

```
contract FactoryRegistry {
   mapping( address =>  address) public proxies;
   mapping( address =>  bool) public isRegistered;
   address[] public allProxies;
  function registerProxy( address implementation) external returns ( address proxy) {
    require(!isRegistered[ msg. sender], "Already␣registered");
     bytes memory bytecode = type(AuthenticatedProxy).creationCode;
     bytes32 salt = keccak256(abi.encodePacked( msg. sender, implementation));
    assembly {
      proxy :=  create2(0,  add(bytecode, 32),  mload(bytecode), salt)
    }
    proxies[ msg. sender] = proxy;
    isRegistered[ msg. sender] = true;
    allProxies.push(proxy);
    AuthenticatedProxy(proxy).initialize( msg. sender, implementation);
  }
}
```

Fig. 9. Code snippet of a Centralized Registry pattern, managing deployed proxy contracts with centralized registration and access control.

Keyless Singleton Pattern. Some dapps require deploying contract instances to the deterministic address across multiple blockchains, such as contracts adhering to the EIP-1820 (Universal Registry Contract) [5]. This pattern, proposed in ERC-2470 (Singleton Factory) [10], involves adopting Nick's method [20] to deploy factory contracts. Using create2 factories (as illustrated in Figure 10), developers can deploy contract instances with the same initialization code to the same address on different chains. The contract's address is determined by the factory address, salt, and initcode; with the factory address constant across chains, using the same salt ensures identical deployed contract addresses.

```
contract SingletonFactory {
  function deploy( bytes memory _initCode,  bytes32 _salt) public returns ( address) {
     address addr;
    assembly {
      addr :=  create2(0,  add(_initCode, 0x20),  mload(_initCode), _salt)
    }
    require(addr !=  address(0), "Deploy␣failed");
    return addr;
  }
  function computeAddress( bytes memory _initCode,  bytes32 _salt) public view returns ( address) {
    return  address( uint160( uint(keccak256(abi.encodePacked(
       byte(0xff),
       address(this),
      _salt,
      keccak256(_initCode)
    )))));
  }
}
```

Fig. 10. Code snippet of Keyless Singleton pattern, enabling deterministic cross-chain deployment using CREATE2 with predictable addresses.

Salted Address Pattern. Some deployers prefer the deployed contract's address not to depend on the hash of the initialization code. For this purpose, the EIP-3171 [3] introduced the Create3 Factory pattern (shown in Figure 11), which ensures that the deployed contract's address is determined solely by the deployer's address and the salt, independent of initialization code. Compared to the Keyless Singleton Pattern, this approach allows

Just Accepted

deployers to conveniently redeploy contract instances with different implementations to the same address across multiple blockchains while retaining predictability.

```
contract Create3Factory {
  function deploy( bytes32 salt,  bytes memory creationCode) external payable returns ( address) {
     bytes memory proxyCreationCode = abi.encodePacked(
      hex"67363d3d37363d34f03d5260086018f3",
      creationCode
    );
     address proxy;
    assembly {
      proxy :=  create2(0,  add(proxyCreationCode, 0x20),  mload(proxyCreationCode), salt)
    }
    require(proxy !=  address(0), "Create3: deployment failed");
    return getDeployed( msg. sender, salt);
  }
  function getDeployed( address deployer,  bytes32 salt) public pure returns ( address) {
     address proxy =  address( uint160( uint(keccak256(abi.encodePacked(
      hex"ff", deployer, salt, keccak256(hex"67363d3d37363d34f03d5260086018f3")
    )))));
    return  address( uint160( uint(keccak256(abi.encodePacked(hex"d694", proxy, hex"01")))));
  }
}
```

Fig. 11. Code snippet of Salted Address pattern (Create3), ensuring deployed contract addresses depend only on deployer and salt, not initialization code.

Just Accepted

Metamorphic Factory Pattern. Immutable code is considered to be the cornerstone of a stable blockchain ecosystem. However, the emergence of the Metamorphic Factory Pattern disrupts this rule. It uses a metamorphic factory to smoothly upgrade factory-deployed contracts without changing the deployed contract address, offering viable support for maintenance, fixes, and upgrades across multiple chains. Unlike the contract upgrade standards specified by protocols like ERC-1822 (Universal Upgradeable Proxy Standard) [6], contracts deployed by a metamorphic factory, once upgraded, delete all existing storage and replace the existing code with the new implementation code. Figure 12 presents a concise implementation of the metamorphic pattern. The *deployMetamorphic()* function utilizes the *create2* opcode to deploy TransientFactory. Within the temporary factory's constructor, first deploy the metamorphic contract using the create opcode, followed by the execution of a self-destruct operation, resetting the transient factory's nonce. When the metamorphic contract needs to be replaced, first execute the *selfdestruct* operation on it. Then call the *deployMetamorphic()* again, and make sure we pass in the same salt as before. The TransientFactory redeploys the new metamorphic contract code to the same address to achieve the seamless contract replacement. This pattern can be exploited for malicious activities. Section 3.10 details its attack vectors and countermeasures.

Answer to RQ4: We systematically identify six distinct factory-based patterns through comprehensive analysis of factory contract implementations: Template Configuration Pattern, Proxy Delegation Pattern, Centralized Registry Pattern, Keyless Singleton Pattern, Salted Address Pattern, and Metamorphic Factory Pattern. These patterns represent fundamental approaches to contract deployment, gas optimization, management efficiency, cross-chain deployment, and contract upgradeability.

```
contract MetamorphicContractFactory {
   bytes private transientFactoryCode;
   mapping( address =>  bytes) private initCodes;
  function deployMetamorphic( bytes32 salt, bytes metamorphicInitCode) external payable {
     address tempFactory;
    assembly {
      let d:= add(0x20, metamorphicInitCode)
      let s:= mload(metamorphicInitCode)
      tempFactory:= create2(callvalue,d,s,salt)
    }
    initCodes[tempFactory]=metamorphicInitCode;
  }
  function getInitCode() external view returns ( bytes) {
    return initCodes[ msg. sender];
  }
}
contract tempFactory {
  constructor() public payable {
     bytes initCode = IFactory( msg. sender).getInitCode();
    assembly {
      let d :=  add(0x20,initCode)
      let s :=  mload(initCode)
      metamorphicAddr :=  create(callvalue,d,s)
    }
    selfdestruct(metamorphicAddr);
  }
}
```

Fig. 12. An implementation of metamorphic pattern.

Just Accepted

3.10 RQ5: Attack Vector and Security Issue Analysis

To assess the security landscape of factory contracts, we conduct a systematic analysis to identify and categorize potential Attack Vectors and Security Issues.[2] Our methodology combines analysis of existing security incident reports and Ethereum Improvement Proposals (EIPs) with comprehensive source code examination, involving review of documented security incidents, analysis of EIP specifications that introduce new deployment mechanisms, and static analysis of factory contract implementations to identify potential vulnerability patterns.

Based on our analysis, we identify two primary attack vectors and three critical security issues in factory contract implementations as shown in Table 4. The following sections provide detailed analysis of each category, including technical analysis, real-world case studies, and countermeasures.

Mutable Code: This attack vector exploits the CREATE2 opcode's deterministic address generation combined with the SELFDESTRUCT operation to achieve contract mutability. Attackers deploy a contract at a specific address, destroy it via SELFDESTRUCT, then redeploy different code at the same address. This technique enables malicious actors to bypass security audits by initially deploying benign contracts that are later replaced with malicious versions.

Case Study. The Tornado Cash governance attack exemplifies the Mutable Code vector [19] and proceeds as follows (also shown in Figure 13): ① **Benign proposal setup via transient factory.** The attacker deploys a transient factory using *CREATE2*, fixing the factory address by *(deployer, salt, keccak256(init_code))*. The factory deploys a seemingly benign governance proposal at a deterministic address; ② **Governance approval.** The community/governance process reviews and approves the benign-looking proposal, and the address is recorded as the executable proposal; ③ **Code removal.** The attacker triggers *SELFDESTRUCT* on both the transient factory

[2]In our study, Attack Vector refers to the specific techniques or methods that malicious actors employ to exploit factory contract mechanisms for unauthorized purposes, involving manipulation of deployment patterns or contract behavior. Security Issue refers to implementation vulnerabilities or coding practices in factory contracts that could lead to unintended consequences or compromise system security.

Table 4. Classification of Factory Contract Attack Vectors and Security Issues (Type: vulnerability classification, Name: specific vulnerability identifier, Code Analysis Level: analysis granularity required for identification)

Type	Name	Code Analysis Level
Attack Vector	(1) Mutable Code	Bytecode
	(2) Address Spoofing Attack	Bytecode
Security Issue	(1) Unhandled Low-Level Contract Creation	Source Code
	(2) Unexpected Ownership Transfer	Source Code
	(3) Unverified Master Contract	Source Code

and the benign proposal, deleting their code while preserving addresses; ④ **Factory re-deployment.** Using the same *salt* and *init_code*, the attacker re-deploys the transient factory at the original factory address via *CREATE2* (nonce reset); ⑤ **Malicious replacement.** The re-deployed factory creates a malicious contract at the original proposal address, replacing the benign code with attack logic; ⑥ **Execution and impact.** When governance executes the approved proposal address, it now points to malicious code, enabling fund drainage and unauthorized actions.

Countermeasures. Defending against Mutable Code attacks requires a multi-layered approach: (i) **Contract Code Analysis** - Before interacting with any contract, especially in governance or high-value transactions, verify that the target contract does not contain SELFDESTRUCT opcodes or unreachable SELFDESTRUCT calls through DELEGATECALL operations. This can be accomplished through bytecode analysis tools or by requiring contracts to undergo formal verification processes that explicitly check for mutability patterns. (ii) **Deployment Chain Validation** - Implement comprehensive validation of the entire contract deployment chain, from the target contract back to its ultimate deployer. This involves tracking whether any contract in the deployment hierarchy is created using CREATE2, and if so, documenting the salt and bytecode parameters used. Governance systems should maintain an immutable record of these parameters to detect any attempts at redeployment. (iii) **Runtime Code Integrity Checks** - Before executing any external call or delegating authority to a contract, verify its current extcodehash matches the expected value from the initial deployment. This check should be performed dynamically, as static analysis alone cannot prevent post-deployment code changes. Smart contracts and DApps should implement periodic integrity verification, especially for critical operations involving significant value transfers or governance decisions.

Address Spoofing Attack: This attack vector leverages CREATE2's deterministic address computation to bypass wallet security alerts that warn users about interactions with known malicious contracts. Attackers precompute addresses that appear legitimate or similar to trusted contracts, then deploy malicious contracts at these addresses [31, 56].

Case Study. The Address Spoofing Attack has gained prominence in recent wallet draining campaigns, where malicious actors systematically exploit CREATE2's deterministic address generation to circumvent security measures [31, 56]. In these sophisticated attacks, criminals first identify popular and trusted DeFi protocols, then use computational methods to generate thousands of potential addresses that visually resemble legitimate contract addresses. For instance, attackers might target a well-known decentralized exchange contract whose address begins with "0x1234..." and use CREATE2 with various salt values to generate addresses like "0x1234abcd..." or "0x1235defg..." that appear similar at first glance. The attack process involves setting up factory contracts specifically designed to deploy malicious contracts at these carefully crafted addresses. When users encounter these addresses in their wallet interfaces, the visual similarity to trusted contracts can bypass both automated security systems that rely on blacklist matching and human verification processes. The malicious contracts deployed at these addresses typically contain functions that appear legitimate but include hidden backdoors for

Just Accepted

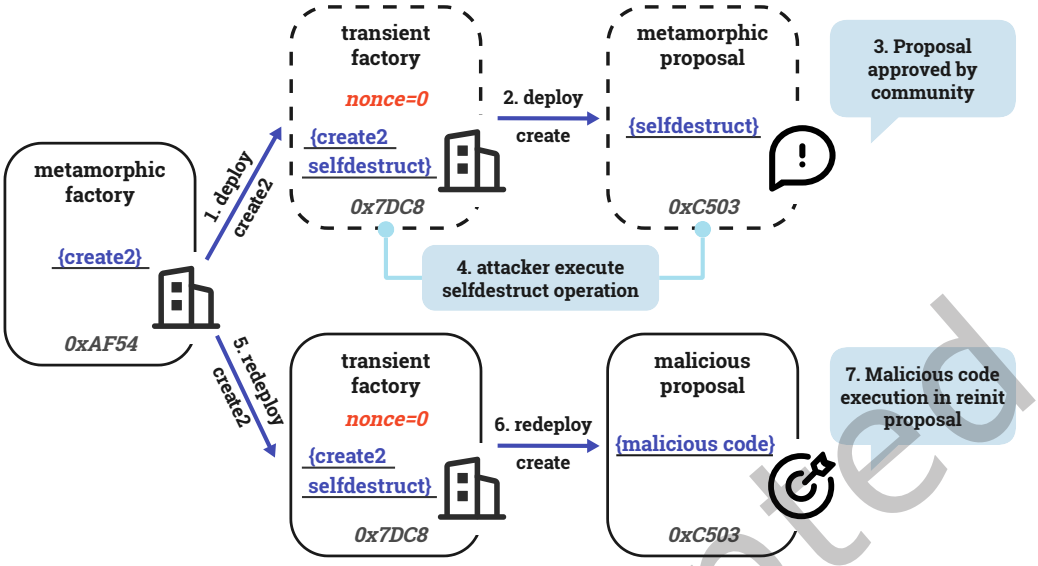

Fig. 13. The process of Tornado Cash DAO attack.

Just Accepted

fund extraction. Some advanced variants of this attack even implement time-based triggers, where the malicious functionality only activates after a certain period, making initial security audits ineffective.

Countermeasures. Defending against Address Spoofing attacks requires a comprehensive strategy that goes beyond simple address comparison: (i) **Enhanced Address Verification Systems** - Wallet providers and DeFi interfaces should implement advanced address verification that examines entire addresses rather than just prefixes or suffixes. This includes maintaining comprehensive databases of verified contract addresses with their associated metadata, implementing fuzzy matching algorithms that can detect visually similar addresses, and providing clear visual indicators when users interact with contracts that have addresses similar to known legitimate ones. Additionally, implementing address reputation systems that track the deployment history and behavioral patterns of contracts can help identify suspicious addresses. (ii) **User Education and Interface Improvements** - Develop educational initiatives to help users understand the risks associated with CREATE2-based address manipulation, including providing clear guidelines on how to verify contract legitimacy beyond address appearance. Wallet interfaces should display full addresses prominently rather than truncating them, implement visual warnings when interacting with recently deployed contracts or contracts with similar addresses to known protocols, and provide easy access to contract verification status and deployment information. (iii) **Behavioral Analysis and Detection** - Implement real-time monitoring systems that analyze contract deployment patterns to identify potential Address Spoofing attempts. This includes tracking the rapid deployment of multiple contracts with similar addresses, monitoring for patterns where contracts are deployed specifically to mimic existing successful protocols, and implementing machine learning algorithms that can detect anomalous address generation patterns that may indicate malicious intent.

Unhandled Low-Level Contract Creation: This security issue occurs when factories use CREATE or CREATE2 opcodes within inline assembly without properly validating the deployment result. Unlike high-level deployment using the 'new' keyword which reverts on failure, low-level opcodes return zero address on failure

```
contract Create2Factory {
   mapping( address =>  bool) deployed;
  function safeCreate2( bytes32 salt, bytes code) external payable {
     address targetAddr =  address( uint160( uint256(keccak256(abi.encodePacked(hex"ff",
         address(this),salt,keccak256(abi.encodePacked(code)))))));
    require(!deployed[targetAddr]);
    assembly {
      let d :=  add(0x20, code)
      let s :=  mload(code)
      newAddr :=  create2(callvalue,d,s,salt)
    }
    require(newAddr == targetAddr);
    deployed[targetAddr] = true;
  }
}
```

Fig. 14. Example of factory contract using inline assembly with CREATE2 opcode for deployment.

while allowing transaction execution to continue. This can result in permanent loss of funds if Ether is attached to failed deployments.

Case Study. Unhandled Low-Level Contract Creation represents a critical vulnerability that affects numerous factory contracts in production environments, leading to significant financial losses and operational failures. This issue primarily manifests when developers use inline assembly with CREATE or CREATE2 opcodes without implementing proper failure handling mechanisms. Unlike Solidity's high-level 'new' keyword which automatically reverts on deployment failures, low-level opcodes return a zero address (0x0) when deployment fails while allowing transaction execution to continue. The vulnerability becomes particularly dangerous when combined with value transfers, as Ether sent to failed deployments can be permanently lost. A notable real-world example occurs in several DeFi factory contracts where developers attempt to optimize gas costs by using inline assembly for contract deployment. In one documented case, a factory contract designed to deploy lending pool contracts experienced multiple deployment failures due to out-of-gas conditions during periods of network congestion. Because the factory code lacked proper validation of the returned address from the CREATE2 operation, the contract continued execution and marked the zero address as a successfully deployed lending pool in its registry. Subsequent attempts by users to interact with these "deployed" contracts resulted in failed transactions, locked funds, and significant user experience degradation. The situation is exacerbated by the factory's value transfer mechanism, which sends initialization Ether to what it believes are newly deployed contracts but are actually null addresses, resulting in irreversible fund losses. Figure 14 illustrates a more robust approach to this challenge, showing how proper validation can prevent such vulnerabilities. The example demonstrates essential safety checks including address validation, pre-deployment verification, and post-deployment confirmation that must be implemented when using low-level deployment operations.

Countermeasures. Preventing Unhandled Low-Level Contract Creation vulnerabilities requires implementing comprehensive validation and error handling mechanisms throughout the deployment process: (i) **Mandatory Return Value Validation** - Always validate that the address returned from CREATE or CREATE2 operations is not the zero address (0x0). This can be implemented through explicit require statements such as 'require(deployedAddress != address(0), "Contract deployment failed")' immediately after the assembly block. Additionally, implement address validation that checks the returned address matches expected patterns for valid Ethereum addresses, ensuring it falls within valid ranges and is not a reserved address. Consider implementing custom error types that provide detailed failure information to help with debugging and incident response. (ii) **Deployment Integrity Verification** - After successful address validation, perform comprehensive integrity checks to verify the contract is actually deployed correctly. This includes verifying that the deployed address contains actual bytecode using 'deployedAddress.code.length > 0' checks, confirming that the deployed bytecode matches the

Just Accepted

```
abstract contract Ownable is Context {
   address private _owner;
  constructor() {
    transferOwnership( msg. sender);
  }
  function transferOwnership( address newOwner) public onlyOwner {
    require(newOwner !=  address(0));
     address oldOwner = _owner;
    _owner = newOwner;
  }
}
contract NFTSample is Ownable,ReentrancyGuard {
  constructor() {
    maxBatchSize = 8;
    collectionSize = 16384;
    _setDefaultRoyalty(_owner, 500);
    operatorFilteringEnabled = true;
  }
  function withdrawMoney() onlyOwner {...}
}
```

Fig. 15. Example of unexpected ownership transfer in factory-deployed contracts.

Just Accepted

expected bytecode hash to prevent deployment of incorrect contract versions, and implementing pre-computed address validation for CREATE2 operations by comparing the returned address with the deterministically calculated address. For critical applications, consider implementing additional verification through external code hash comparisons or checksum validation. (iii) **State Management and Rollback Mechanisms** - Implement comprehensive state management that properly handles deployment failures and prevents inconsistent contract states. This includes maintaining deployment status maps that are only updated after successful verification of all deployment steps, implementing atomic deployment patterns where all related state changes are bundled together and either succeed or fail as a unit, and providing rollback mechanisms for partial deployment failures. Additionally, implement event logging for all deployment attempts, including failures, to enable proper monitoring and incident response. Consider implementing circuit breaker patterns for factory contracts experiencing high deployment failure rates to prevent cascade failures and resource exhaustion.

Unexpected Ownership Transfer: This security issue arises when factory-deployed contracts incorrectly use msg.sender in constructors for ownership assignment. Since factories deploy contracts on behalf of users, msg.sender refers to the factory address rather than the intended owner, potentially causing loss of access to critical contract functions.

Case Study. The Unexpected Ownership Transfer vulnerability represents one of the most pervasive yet subtle security issues affecting factory-deployed contracts, particularly those utilizing standard ownership patterns from libraries like OpenZeppelin. This vulnerability manifests in numerous real-world deployments, causing significant operational disruptions and financial losses across various DeFi and NFT projects. The core issue arises from a fundamental misunderstanding of execution context during contract deployment: when a factory contract deploys a new contract, the msg.sender during the constructor execution refers to the factory's address rather than the external account that initiated the transaction. This seemingly minor detail has far-reaching implications for contracts that rely on msg.sender for access control initialization. A notable example occurs in several NFT marketplace factories where thousands of contracts are deployed with incorrect ownership assignments. In one documented incident, a popular NFT creation platform deploys over 10,000 NFT contracts through its factory system, with each contract inheriting from OpenZeppelin's Ownable pattern as shown in Figure 15. The contracts are designed to allow creators to mint NFTs and withdraw accumulated sales revenue through the withdrawMoney() function protected by the onlyOwner modifier. However, due to the Unexpected Ownership

Transfer vulnerability, the factory contract becomes the owner of all deployed NFT contracts instead of the intended creators. This means that thousands of NFT creators lose access to critical functions in their contracts, including the ability to withdraw funds from sales, modify contract parameters, or execute administrative functions. The situation requires extensive manual intervention and gas-expensive ownership transfer transactions to restore proper access control. In some cases, the factory contract itself lacks proper administrative functions to transfer ownership back to the intended users, requiring complex upgrade procedures and community coordination to resolve. The vulnerability becomes particularly problematic during high-value NFT drops where significant funds are locked in contracts with inaccessible withdrawal functions.

Countermeasures. Preventing Unexpected Ownership Transfer vulnerabilities requires implementing robust ownership initialization patterns that account for factory deployment contexts: (i) **Parameter-Based Ownership Initialization** - The most effective approach is to modify contract constructors to accept ownership parameters explicitly rather than relying on msg.sender. This involves refactoring constructors to include address parameters for the intended owner, such as ‘constructor(address initialOwner)‘, and ensuring all inherited contracts (like OpenZeppelin’s Ownable) are properly initialized with these parameters. Factory contracts should be designed to pass tx.origin or explicitly provided owner addresses to deployed contracts during construction. This approach provides clear ownership intent and eliminates ambiguity about who should control the deployed contract. Additionally, implement validation checks in constructors to ensure that provided owner addresses are not zero addresses and represent valid externally owned accounts or approved contract addresses. (ii) **Post-Deployment Ownership Transfer Patterns** - For situations where constructor modification is not feasible, implement systematic post-deployment ownership transfer mechanisms. This includes designing factory contracts with built-in ownership transfer capabilities that automatically call transferOwnership(tx.origin) immediately after successful contract deployment, implementing batch ownership transfer functions that can correct ownership for multiple contracts simultaneously, and providing emergency recovery mechanisms for cases where initial ownership assignment fails. Consider implementing time-locked transfer mechanisms that allow a grace period for ownership corrections. Additionally, maintain comprehensive logging of all ownership transfer operations to enable audit trails and facilitate troubleshooting.

```
import "./proxy/Clones.sol";
contract NinfaFactory is AccessControl {
  using Clones for  address;
   mapping( address => bool) collectionWhitelist;
  function cloneCollection( address master, bytes32 salt, bytes data) public {
    require(hasRole(MINTER_ROLE,  msg. sender));
    require(collectionWhitelist[master]==true);
    clone = master.cloneDeterministic(salt);
    // bytes4(keccak256('initialize(bytes)'))
    ( bool success,) = clone.call(abi.encodeWithSelector(0x439fab91,data));
    require(success);
    emit NewClone(clone,  msg. sender);
  }
}
```

Fig. 16. Example of master contract validation using whitelist and initialization verification.

Unverified Master Contract: This security issue affects factories implementing the Proxy Delegation Pattern, where externally provided master contract addresses are not properly validated before proxy creation. Malicious or incompatible master contracts can compromise the security and functionality of all deployed proxies.

Case Study. Figure 16 is a simplified version of the NinfaFactory contract on the Ethereum mainnet. It serves as a factory for creating NFT distribution contracts. The contract imports the Clones library provided by OpenZeppelin to deploy minimal proxies. The cloneCollection function takes the master contract’s address as a parameter. To

Just Accepted

ensure the validity of the master address, it uses require(collectionWhitelist[master]==true) to check if the master contract to be cloned is present in the whitelist. Additionally, the contract uses clone.call(func_selector) to invoke a method of the clone and uses require(success) to verify the result of the function call.

Countermeasures. Defending against unverified master contract vulnerabilities requires implementing comprehensive validation mechanisms that ensure both the integrity and compatibility of master contracts: (i) **Whitelist-Based Access Control** - Implement a robust whitelist system that maintains a registry of approved master contracts, similar to the NinfaFactory example. This system should include role-based access controls (such as ADMIN_ROLE) for adding contracts to the whitelist, regular auditing processes to review and update the whitelist based on security assessments, and event logging for all whitelist modifications to maintain transparency. The whitelist should store additional metadata about each approved contract, including deployment date, audit results, and compatibility version information. (ii) **Dynamic Contract Validation** - Before deploying any proxy, perform comprehensive validation of the master contract including verification that the master address contains actual contract code using extcodesize checks, validation that the master contract implements expected interfaces through ERC-165 or similar standards, and testing critical functions to ensure they exist and behave as expected. Implement automated compatibility testing that validates the master contract against known function signatures and expected behaviors. (iii) **Initialization and Post-Deployment Verification** - After deploying each proxy, implement comprehensive verification processes to ensure proper initialization. This includes validating that the initialization function call succeeds and produces expected state changes, implementing comprehensive error handling and revert mechanisms if initialization fails, and performing post-deployment verification that the proxy correctly delegates to the master contract. Additionally, consider implementing time-locked deployment processes where proxies are deployed in a preliminary state and only activated after successful verification, providing an opportunity to catch and remediate issues before the contracts become operational.

Answer to RQ5: Our analysis identifies two attack vectors (Mutable Code and Address Spoofing Attack) that exploit factory contract mechanisms for malicious purposes, and three security issues (Unhandled Low-Level Contract Creation, Unexpected Ownership Transfer, and Unverified Master Contract) that represent implementation vulnerabilities in factory contracts requiring systematic mitigation strategies.

4 Discussion

In this section, we explore some practical implications of our findings, discuss potential threats to their validity, and outline directions for future work.

4.1 Calls to Action

(i) For Decentralized Governance Committees. The Tornado Cash incident shows that benign-looking proposals can be swapped via metamorphic techniques after approval. Committees should reject proposals whose deployment chain contains *selfdestruct*/*delegatecall* or whose code provenance cannot be verified end to end (factory address, salt, and init_code hash should all be auditable and consistent). Proposers ought to attest and freeze these deployment parameters on-chain prior to voting, and the same values should be verified again at execution. A time-locked execution with a short post-approval audit window further reduces risk by allowing re-verification that the code at the proposal address still matches the reviewed version.

(ii) For Wallet Providers, Security Tools, and dApp Frontends. Address Spoofing via *CREATE2* abuses approvals to undeployed "predicted" addresses. Wallets should flag and require explicit confirmation whenever an approval/permit targets a spender with no code (*EXTCODESIZE=0*), and default such approvals to bounded amount and duration. Risk context should be surfaced: if the spender is a predicted *CREATE2* address from a known factory, show the factory address and, when available, the attested init_code hash; warn or block when provenance

Just Accepted

is absent. Frontends should avoid asking users to approve undeployed spenders, prefer deploy-first then approve-after-code-is-present, or bind approvals to code identity by displaying and verifying keccak256(runtime_code). When using factories, pre-commit deployment parameters (factory, salt, init_code hash), emit attestation events, and provide verification links. Both wallets and frontends should offer first-class flows to review and revoke high-risk approvals (including undeployed or recently deployed spenders) and integrate allowlists/denylists curated by reputable communities.

(iii) For Contract Deployers. Weigh factory overheads versus reuse benefits. Proxy delegation (Section 3.9) amortizes code storage but requires whitelist management and initialization verification. Account for constructor context as in Section 3.10: initialize ownership via explicit parameters or perform an immediate post-deploy transfer, rather than relying on *msg.sender*.

(iv) For Contract Developers. Harden low-level creation logic by validating returned addresses, handling create/create2 failures, and emitting events with salts and code hashes to aid traceability. When cloning, verify master contracts via whitelists and live calls (Section 16) and fail fast on mismatches. Prefer vetted libraries (OpenZeppelin's *Clones*, *Ownable*, *ReentrancyGuard*), and document deployment chains to aid audits and community review.

4.2 Threats to Validity

Construct Validity. The research on factory-based patterns (Section 3.9) and the identification of potential security risks (Section 3.10) involve manual assessments. We mitigate this threat as follows. We establish predefined auditing guidelines for reviewers (available in our repository [26]). These guidelines decompose factory functionalities into orthogonal components and finer-grained sub-functionalities. After iterative validation, we identify nine sub-functionalities. During a manual re-audit, reviewers examine each sub-functionality individually and record concrete implementation methods together with related contract components. We then conduct a three-round iterative review to derive patterns and security risks, with three trained auditors performing independent reviews and voting in each round, and regular discussions with a domain expert. Each round supplements or restructures previous results, improving completeness and consistency. The review artifacts are publicly available [26].

Internal Validity. We use Slither to compile and analyze contract source code. A total of 139 factory contracts fail to compile, which could affect measurements. We found that 79 of these failures are due to unresolved inline assembly. To enable compilation without biasing results, we minimally adjusted or stubbed inline assembly that is unrelated to the measured aspects, ensuring the core semantics relevant to our analyses remain intact.

External Validity. External threats concern the generalizability and representativeness of the selected data. We focus on two major EVM-compatible chains (Ethereum and Polygon) because comprehensive contract data is publicly available (e.g., via BigQuery), which may limit immediate generalizability to other ecosystems. We plan to extend our analysis to additional EVM-compatible chains as their datasets mature and become more reproducible. As the ecosystem evolves, new factory-related deployment patterns (e.g., CREATE3 factories [3]) and security considerations continue to emerge; we complement our taxonomy and analyses accordingly. To minimize bias, we curate a wide-ranging and reproducible dataset and release both raw and curated factory-contract datasets to facilitate reuse and independent validation.

5 Related Work

5.1 Empirical Studies of Smart Contracts

Some empirical studies delve into smart contracts. Relevant research concentrates on contract deployment [30, 41] and factory-related patterns [55, 59]. Chaliasos et al. [30] investigate the prevalence of *create* / *create2* instructions. They categorize inline assembly fragments into 10 classes, finding that system operations accounted for 75.06% of total fragments, with *create2* being the most common at 45.09%. Fröwis et al. [41] explore the impact and

Just Accepted

usage of the *create2* instruction, finding that 47% of contract accounts use *create2* for deployment after the Constantinople upgrade. They also provide a corresponding detection algorithm and identify 41 metamorphic code accounts. Sun et al. [59] discuss the top-reused factory interface and extract two factory patterns in the token exchange and issuance domains. However, these studies only focus on a single aspect of factory contracts, covering types, patterns, and application domains, without specific attention to factory-based deployments themselves. This limitation leaves room for further exploration. Our empirical study comprehensively summarizes multiple dimensions, enhancing the understanding of factory contracts in practical development.

Additionally, there are empirical studies focusing on specific features, patterns, and applications of smart contracts [38, 42, 45, 47, 50, 54, 63]. Liu et al. [50] empirically evaluate the real-world impact of EIP-1559 on transaction fee dynamics, waiting times, and consensus security. Meisami et al. [45] pioneer the development of a taxonomy for upgradeable contracts and explore the significance, patterns, and security issues. Fang et al. [38] employ NLP and cluster analysis to categorize four usages of custom modifiers. Qian et al. [54] conduct a review of various DeFi attacks and systematically evaluate state-of-the-art smart contract security detection tools. Our empirical study systematically discusses various dimensions of factory contracts in a progressive manner.

5.2 Security Analysis of Smart Contracts

Given the relevance of smart contracts to finance, substantial research [29, 37, 43, 44, 48, 52, 62] is conducted in the field of smart contract security analysis. VetSC [37] automates DApp safety vetting through UI-guided semantic extraction, graph generation and model checking, revealing security risks like expired tickets and double voting. VRust [36] pioneers a vulnerability detection framework tailored for Solana smart contracts, introducing static analysis to validate untrusted input accounts specific to stateless programming model. SmartDagger [48] provides a machine-learning-based semantic recovery mechanism, supporting the detection of cross-contract reentrancy vulnerabilities at the bytecode level. MadMax [44] and eTainter [43] tackle gas-related security issues.

However, with the diversification of contract patterns and increased composability, these tools may struggle to identify vulnerabilities that arise in specific contexts, leaving this aspect relatively unexplored. USCHunt [45] focuses on security issues with proxy-based upgradeable patterns, identifying six types of vulnerabilities and discovering 2546 insecure contracts across blockchains. Building on this direction, an ecosystem-scale empirical study of proxy smart contracts further characterizes their prevalence, patterns, and security implications across the Ethereum ecosystem [64]. Additionally, SPCon [49] and SoMo [38] consider security issues related to function modifiers. Several studies are dedicated to designing and implementing frameworks for smart contract analysis [39, 40, 57]. To the best of our knowledge, these tools do not specifically address security issues in the factory context. Our work fills this gap by thoroughly examining four security issues related to factory contracts, providing case studies and outlining their triggering conditions.

5.3 Factory Contracts and Deployment Mechanisms

Factory contracts and deployment mechanisms have received increasing attention due to their impact on determinism, upgradeability, and ecosystem-scale deployment practices. Empirical and measurement studies document how low-level deployment instructions and ecosystem conventions shape development patterns and risks. Fröwis and Böhme systematically analyze deployments using *create2* and report its role in enabling metamorphic behaviors in the wild [41]. Chaliasos et al. characterize the prevalence of *create*/*create2* in inline assembly and provide taxonomy and tooling implications for safe usage [30]. Sun et al. mine factory interfaces and distill common factory patterns seen across major domains [59]. At the ecosystem scale, USCHunt uncovers systemic issues in proxy-based upgradeability [45], and subsequent work extends this line with an ecosystem-scale empirical study of proxy contracts on Ethereum [64]. From the perspective of deployment engineering, research has explored code generation and deployment pipelines for smart contracts [61], and examined gas modeling and

Just Accepted

optimization that directly influence deployment choices and factory adoption [28, 46, 58]. Our work complements these directions with a comprehensive, data-driven analysis of factory prevalence, functionality patterns, and security risks in practice.

6 Conclusion

This empirical study offers the first ecosystem-scale view of smart-contract factory activity across Ethereum and Polygon. We implement a bytecode-level factory contract detector, validate it on a ground-truth benchmark, and apply it to 434,542,165 deployed runtimes to surface 120,204 Ethereum factories and 69,258 Polygon factories together with their factory-created contracts. Building on this detector-driven corpus, we uncover three complementary insights. Longitudinal measurements show factories have minted over 90% of contracts since 2020, with creation volume concentrated in a small set of high-throughput deployers. Clustering of 3,000 verified factories reveals four principal application domains spanning finance, infrastructure, proxy upgradeability, and NFT creation. Finally, semantic inspection distills six recurring implementation patterns and highlights factory-centric attack vectors and security issues that warrant targeted mitigations. These findings equip developers with data-driven guidance on when to employ factory patterns, assist auditors in reasoning about factory provenance and governance, and motivate researchers to pursue defenses.

Acknowledgments

The authors would like to thank the editors and the anonymous reviewers for their insightful comments and suggestions, which can substantially improve the quality of this work.

References

[1] 2018. EIP-1014: Skinny CREATE2. https://eips.ethereum.org/EIPS/eip-1014 (Accessed on 10/25/2024).
[2] 2018. ERC-1167: Minimal Proxy Contract. https://eips.ethereum.org/EIPS/eip-1167 (Accessed on 10/25/2024).
[3] 2018. ERC-3171: Add Create3 Opcode. https://github.com/ethereum/EIPs/pull/3171 (Accessed on 10/25/2024).
[4] 2019. 0age: Metamorphic. https://github.com/0age/metamorphic (Accessed on 10/25/2024).
[5] 2019. ERC-1820: Pseudo-introspection Registry Contract. https://eips.ethereum.org/EIPS/eip-1820 (Accessed on 10/25/2024).
[6] 2019. ERC-1822: Universal Upgradeable Proxy Standard. https://eips.ethereum.org/EIPS/eip-1822 (Accessed on 10/25/2024).
[7] 2019. Factory Contract. https://research.csiro.au/blockchainpatterns/general-patterns/contract-structural-patterns/factory-contract/ (Accessed on 10/25/2024).
[8] 2019. Openzeppelin: Clones. https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/proxy/Clones.sol (Accessed on 10/25/2024).
[9] 2019. Openzeppelin: Create2. https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/Create2.sol (Accessed on 10/25/2024).
[10] 2020. ERC-2470: Singleton Factory. https://eips.ethereum.org/EIPS/eip-2470 (Accessed on 10/25/2024).
[11] 2020. Openzeppelin: ProxyFactory. https://github.com/OpenZeppelin/openzeppelin-sdk/blob/master/packages/lib/contracts/upgradeability/ProxyFactory.sol (Accessed on 10/25/2024).
[12] 2020. Uniswap V2: Factory Contract. https://etherscan.io/address/0x5c69bee701ef814a2b6a3edd4b1652cb9cc5aa6f#code (Accessed on 10/25/2024).
[13] 2022. Ethereum. https://ethereum.org/en (Accessed on 10/25/2024).
[14] 2022. Etherscan. https://etherscan.io (Accessed on 10/25/2024).
[15] 2022. Solidity Language. https://soliditylang.org (Accessed on 10/25/2024).
[16] 2022. SweetmanTech: NFT-Factory. https://github.com/SweetmanTech/nft-factory (Accessed on 10/25/2024).
[17] 2022. ThirdWeb. https://thirdweb.com (Accessed on 10/25/2024).
[18] 2022. Uniswap. https://uniswap.org (Accessed on 10/25/2024).
[19] 2023. Exploiting governance with metamorphic proposals. https://www.coinbase.com/blog/exploiting-governance-with-metamorphic-proposals (Accessed on 10/25/2024).
[20] 2023. Nick's method — Ethereum Keyless Execution. https://yamenmerhi.medium.com/nicks-method-ethereum-keyless-execution-168a6659479c (Accessed on 10/25/2024).
[21] 2023. Smart contract. https://en.wikipedia.org/wiki/Smart_contract (Accessed on 10/25/2024).

Just Accepted

[22] 2023. Smart Contract Deployment Statistics. https://dune.com/pcaversaccio/smart-contract-deployment-statistics (Accessed on 10/25/2024).
[23] 2024. Bitcoin. https://bitcoin.org/en/ (Accessed on 10/25/2024).
[24] 2024. Ethereum Account. https://ethereum.org/en/developers/docs/accounts/ (Accessed on 10/25/2024).
[25] 2024. Ethereum Virtual Machine. https://en.wikipedia.org/wiki/Ethereum#Virtual_machine (Accessed on 10/25/2024).
[26] 2024. FSCs empirical analysis. https://anonymous.4open.science/r/FSCs-empirical-analysis (Accessed on 10/25/2024).
[27] 2024. Gas. https://en.wikipedia.org/wiki/Ethereum#Gas (Accessed on 10/25/2024).
[28] Elvira Albert, Pablo Gordillo, Alejandro Hernández-Cerezo, Albert Rubio, and Maria Anna Schett. 2022. Super-optimization of Smart Contracts. *ACM Trans. Softw. Eng. Methodol.* (2022). doi:10.1145/3506800
[29] Lexi Brent, Neville Grech, Sifis Lagouvardos, Bernhard Scholz, and Yannis Smaragdakis. 2020. Ethainter: a smart contract security analyzer for composite vulnerabilities. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI*. ACM. doi:10.1145/3385412.3385990
[30] Stefanos Chaliasos, Arthur Gervais, and Benjamin Livshits. 2022. A study of inline assembly in solidity smart contracts. *Proc. ACM Program. Lang.* OOPSLA2 (2022). doi:10.1145/3563328
[31] Check Point Research. 2024. Ethereum's CREATE2: A Double-Edged Sword in Blockchain Security. https://research.checkpoint.com/2024/ethereums-create2-a-double-edged-sword-in-blockchain-security/ (Accessed on 09/16/2025).
[32] Jiachi Chen, Xin Xia, David Lo, John C. Grundy, and Xiaohu Yang. 2021. Maintenance-related concerns for post-deployed Ethereum smart contract development: issues, techniques, and future challenges. *Empir. Softw. Eng.* (2021). doi:10.1007/S10664-021-10018-0
[33] Google Cloud. 2024. BigQuery Public Datasets. https://cloud.google.com/bigquery/public-data/ (Accessed on 10/25/2024).
[34] Google Cloud. 2024. Google BigQuery: Cloud Data Warehouse. https://cloud.google.com/bigquery (Accessed on 10/25/2024).
[35] Google Cloud. 2024. Google BigQuery Ethereum Traces Dataset. https://console.cloud.google.com/bigquery?ws=!1m5!1m4!4m3!1sbigquery-public-data!2scrypto_ethereum!3straces (Accessed on 10/25/2024).
[36] Siwei Cui, Gang Zhao, Yifei Gao, Tien Tavu, and Jeff Huang. 2022. VRust: Automated Vulnerability Detection for Solana Smart Contracts. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS*. https://doi.org/10.1145/3548606.3560552
[37] Yue Duan, Xin Zhao, Yu Pan, Shucheng Li, Minghao Li, Fengyuan Xu, and Mu Zhang. 2022. Towards Automated Safety Vetting of Smart Contracts in Decentralized Applications. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS*. doi:10.1145/3548606.3559384
[38] Yuzhou Fang, Daoyuan Wu, Xiao Yi, Shuai Wang, Yufan Chen, Mengjie Chen, Yang Liu, and Lingxiao Jiang. 2023. Beyond "Protected" and "Private": An Empirical Security Analysis of Custom Function Modifiers in Smart Contracts. In *Proceedings of the 32nd SIGSOFT International Symposium on Software Testing and Analysis, ISSTA*. ACM. doi:10.1145/3597926.3598125
[39] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: a static analysis framework for smart contracts. In *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, WETSEB@ICSE 2019*. IEEE / ACM. doi:10.1109/WETSEB.2019.00008
[40] João F. Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. 2020. SmartBugs: A Framework to Analyze Solidity Smart Contracts. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE*. IEEE. doi:10.1145/3324884.3415298
[41] Michael Fröwis and Rainer Böhme. 2022. Not All Code are Create2 Equal. In *Financial Cryptography and Data Security. FC 2022 International Workshops - CoDecFin, DeFi, Voting, WTSC (Lecture Notes in Computer Science)*. Springer. doi:10.1007/978-3-031-32415-4_32
[42] Enrique Fynn, Alysson Bessani, and Fernando Pedone. 2020. Smart Contracts on the Move. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN*. IEEE. doi:10.1109/DSN48063.2020.00040
[43] Asem Ghaleb, Julia Rubin, and Karthik Pattabiraman. 2022. eTainter: detecting gas-related vulnerabilities in smart contracts. In *ISSTA '22: 31st SIGSOFT International Symposium on Software Testing and Analysis*. ACM. doi:10.1145/3533767.3534378
[44] Neville Grech, Michael Kong, Anton Jurisevic, Lexi Brent, Bernhard Scholz, and Yannis Smaragdakis. 2018. MadMax: surviving out-of-gas conditions in Ethereum smart contracts. *Proc. ACM Program. Lang.* OOPSLA (2018). doi:10.1145/3276486
[45] William Edward Bodell III, Sajad Meisami, and Yue Duan. 2023. Proxy Hunting: Understanding and Characterizing Proxy-based Upgradeable Smart Contracts in Blockchains. In *32nd USENIX Security Symposium, USENIX Security*. USENIX Association. https://www.usenix.org/conference/usenixsecurity23/presentation/bodell
[46] Chunmiao Li. 2021. Gas Estimation and Optimization for Smart Contracts on Ethereum. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE*. IEEE. doi:10.1109/ASE51524.2021.9678932
[47] Zhou Liao, Shuwei Song, Hang Zhu, Xiapu Luo, Zheyuan He, Renkai Jiang, Ting Chen, Jiachi Chen, Tao Zhang, and Xiaosong Zhang. 2023. Large-Scale Empirical Study of Inline Assembly on 7.6 Million Ethereum Smart Contracts. *IEEE Trans. Software Eng.* (2023). doi:10.1109/TSE.2022.3163614
[48] Zeqin Liao, Zibin Zheng, Xiao Chen, and Yuhong Nan. 2022. SmartDagger: a bytecode-based static analysis approach for detecting cross-contract vulnerability. In *ISSTA '22: 31st SIGSOFT International Symposium on Software Testing and Analysis*. ACM. doi:10.1145/3533767.3534222

Just Accepted

[49] Ye Liu, Yi Li, Shang-Wei Lin, and Cyrille Artho. 2022. Finding permission bugs in smart contracts with role mining. In *ISSTA '22: 31st SIGSOFT International Symposium on Software Testing and Analysis*. ACM. doi:10.1145/3533767.3534372

[50] Yulin Liu, Yuxuan Lu, Kartik Nayak, Fan Zhang, Luyao Zhang, and Yinhong Zhao. 2022. Empirical Analysis of EIP-1559: Transaction Fees, Waiting Times, and Consensus Security. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. doi:10.1145/3548606.3559341

[51] Sambit Nayak, Nanjangud C. Narendra, Anshu Shukla, and James Kempf. 2018. Saranyu: Using Smart Contracts and Blockchain for Cloud Tenant Management. In *11th IEEE International Conference on Cloud Computing, CLOUD*. IEEE Computer Society. doi:10.1109/CLOUD.2018.00121

[52] Daniel Perez and Benjamin Livshits. 2021. Smart Contract Vulnerabilities: Vulnerable Does Not Imply Exploited. In *30th USENIX Security Symposium, USENIX Security*. USENIX Association. https://www.usenix.org/conference/usenixsecurity21/presentation/perez

[53] Polygon Technology. 2024. Polygon: Ethereum's Internet of Blockchains. https://polygon.technology/ (Accessed on 10/25/2024).

[54] Peng Qian, Rui Cao, Zhenguang Liu, Wenqing Li, Ming Li, Lun Zhang, Yufeng Xu, Jianhai Chen, and Qinming He. 2023. Empirical Review of Smart Contract and DeFi Security: Vulnerability Detection and Automated Repair. *CoRR* (2023). doi:10.48550/ARXIV.2309.02391 arXiv:2309.02391

[55] Mehdi Salehi, Jeremy Clark, and Mohammad Mannan. 2022. Not so Immutable: Upgradeability of Smart Contracts on Ethereum. In *Financial Cryptography and Data Security. FC 2022 International Workshops - CoDecFin, DeFi, Voting, WTSC (Lecture Notes in Computer Science)*. Springer. doi:10.1007/978-3-031-32415-4_33

[56] ScamSniffer. 2024. Wallet Drainers Start Using CREATE2 to Bypass Wallet Security Alert. https://drops.scamsniffer.io/wallet-drainers-starts-using-create2-bypass-wallet-security-alert/ (Accessed on 09/16/2025).

[57] Sunbeom So, Myungho Lee, Jisu Park, Heejo Lee, and Hakjoo Oh. 2020. VERISMART: A Highly Precise Safety Verifier for Ethereum Smart Contracts. In *2020 IEEE Symposium on Security and Privacy, SP 2020*. IEEE. doi:10.1109/SP40000.2020.00032

[58] Andrea Di Sorbo, Sonia Laudanna, Anna Vacca, Corrado Aaron Visaggio, and Gerardo Canfora. 2022. Profiling gas consumption in solidity smart contracts. *J. Syst. Softw.* (2022). doi:10.1016/J.JSS.2021.111193

[59] Kairan Sun, Zhengzi Xu, Chengwei Liu, Kaixuan Li, and Yang Liu. 2023. Demystifying the Composition and Code Reuse in Solidity Smart Contracts. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE*. ACM. doi:10.1145/3611643.3616270

[60] Etherscan Team. 2024. Etherscan Verified Contracts. https://etherscan.io/contractsVerified (Accessed on 10/25/2024).

[61] Konstantinos Tsiounis and Kostas Kontogiannis. 2022. Goal and Policy Based Code Generation and Deployment of Smart Contracts. In *IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER*. IEEE. doi:10.1109/SANER53432.2022.00145

[62] Yinxing Xue, Mingliang Ma, Yun Lin, Yulei Sui, Jiaming Ye, and Tianyong Peng. 2020. Cross-Contract Static Analysis for Detecting Practical Reentrancy Vulnerabilities in Smart Contracts. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE*. IEEE. doi:10.1145/3324884.3416553

[63] Tingting Yin, Chao Zhang, Yuandong Ni, Yixiong Wu, Taiyu Wong, Xiapu Luo, Zheming Li, and Yu Guo. 2022. An Empirical Study on Implicit Constraints in Smart Contract Static Analysis. In *44th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP)*. IEEE. doi:10.1109/ICSE-SEIP55303.2022.9793963

[64] Mengya Zhang, Preksha Shukla, Wuqi Zhang, Zhuo Zhang, Pranav Agrawal, Zhiqiang Lin, Xiangyu Zhang, and Xiaokuan Zhang. 2025. An Empirical Study of Proxy Contracts at the Ethereum Ecosystem Scale. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2996–3008. doi:10.1109/ICSE55347.2025.00083

[65] Weiqin Zou, David Lo, Pavneet Singh Kochhar, Xuan-Bach Dinh Le, Xin Xia, Yang Feng, Zhenyu Chen, and Baowen Xu. 2021. Smart Contract Development: Challenges and Opportunities. *IEEE Trans. Software Eng.* (2021). doi:10.1109/TSE.2019.2942301

Just Accepted